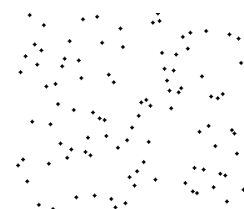


Mergesort: ordenação por intercalação



Considere o problema de ordenação discutido em [outro capítulo](#): rearranjar um vetor $v[0 \dots n-1]$ de tal modo que ele fique em ordem [crescente](#), isto é, de tal modo que tenhamos $v[0] \leq \dots \leq v[n-1]$. O outro capítulo analisou alguns algoritmos simples para o problema. Aqueles algoritmos são [quadráticos](#), ou seja, eles consomem uma quantidade de tempo proporcional a n^2 .

O presente capítulo examina um algoritmo mais sofisticado e muito mais rápido que usa a estratégia da [divisão-e-conquista](#). A ideia é simples: se a primeira metade do vetor já estiver em ordem crescente e a segunda metade também for crescente, então as duas metades podem ser rapidamente intercaladas de modo que o vetor todo fique em ordem crescente.

Sumário:

- [Intercalação de vetores ordenados](#)
- [Intercalação com sentinelas](#)
- [Algoritmo Mergesort](#)
- [Animações do Mergesort](#)
- [Desempenho do algoritmo Mergesort](#)
- [Versão iterativa do Mergesort](#)
- [Animações da versão iterativa do Mergesort](#)

Intercalação de vetores ordenados

Antes de resolver nosso problema principal é preciso resolver o seguinte *problema da intercalação* (= *merge*): dados vetores crescentes $v[p \dots q-1]$ e $v[q \dots r-1]$, rearranjar $v[p \dots r-1]$ em ordem crescente.

p				q-1				q	r-1			
111	333	555	555	777	999	999	999	222	444	777	888	

Seria fácil resolver o [problema](#) da intercalação em tempo proporcional ao quadrado do tamanho do vetor $v[p \dots r-1]$: bastaria ignorar que as duas “metades” já estão ordenadas e usar um dos [algoritmos básicos de ordenação](#). Mas é possível fazer algo bem mais rápido. Para isso, será preciso usar uma “área de trabalho”, digamos $w[0 \dots r-p-1]$, do mesmo tipo e mesmo tamanho que o vetor $v[p \dots r-1]$.

```
// A função recebe vetores crescentes v[p..q-1]
// e v[q..r-1] e rearranja v[p..r-1] em ordem
// crescente.
```

```

static void
intercala (int p, int q, int r, int v[])
{
    int *w; // 1
    w = malloc ((r-p) * sizeof (int)); // 2
    int i = p, j = q; // 3
    int k = 0; // 4

    while (i < q && j < r) { // 5
        if (v[i] <= v[j]) w[k++] = v[i++]; // 6
        else w[k++] = v[j++]; // 7
    } // 8
    while (i < q) w[k++] = v[i++]; // 9
    while (j < r) w[k++] = v[j++]; // 10
    for (i = p; i < r; ++i) v[i] = w[i-p]; // 11
    free (w); // 12
}

```

0:00 / 1:59

A palavra-chave `static` indica que a função `intercala` tem caráter auxiliar, e não deve ser invocada diretamente pelo usuário do algoritmo de ordenação.

Desempenho. A função `intercala` consiste essencialmente em *movimentar* elementos do vetor `v` de um lugar para outro (primeiro de `v` para `w` e depois de `w` para `v`). A função executa

$$2n$$

dessas movimentações, sendo n o tamanho do vetor `v[p..r-1]`. O tempo que `intercala` consome é proporcional ao número de movimentações. Portanto, o consumo de tempo da função é proporcional a n . Assim, o algoritmo é [linear](#).

Exercícios 1

1. Escreva uma função que receba vetores disjuntos `x[0..m-1]` e `y[0..n-1]`, ambos em ordem crescente, e produza um vetor `z[0..m+n-1]` que contenha o resultado da intercalação dos dois vetores dados. Escreva duas versões da função: uma iterativa e uma recursiva.
2. A função `intercala` está correta quando `p` é igual a `q`, ou seja, quando o vetor `v[p..q-1]` está vazio? E quando o vetor `v[q..r-1]` está vazio?
3. Troque as linhas 9 a 11 da função `intercala` pelas duas linhas a seguir. A função continua correta?

```

while (i < q) w[k++] = v[i++];
for (i = p; i < j; ++i) v[i] = w[i-p];

```

4. Troque o bloco de linhas 5 a 8 da função `intercala` pelas linhas abaixo. Critique o efeito da troca.

```

while (i < q && j < r) {
    if (v[i] <= v[j]) w[k++] = v[i++];
    if (v[i] > v[j]) w[k++] = v[j++]; }

```

5. Troque o bloco de linhas 3 a 10 da função `intercala` pelas linhas abaixo. A função continua correta?

```

i = p; j = q;
for (k = 0; k < r-p; ++k) {
    if (j >= r || (i < q && v[i] <= v[j]))
        w[k] = v[i++];
    else
        w[k] = v[j++]; }

```

6. Na função *intercala*, troque o bloco de linhas 5 a 10 pelas linhas abaixo. A função continua correta?

```
while (k < r-p) {
    while (i < q && v[i] <= v[j])
        w[k++] = v[i++];
    while (j < r && v[j] <= v[i])
        w[k++] = v[j++]; }
```

7. INVARIANTES. Quais são os [invariantes](#) do primeiro `while` na função *intercala*?

8. Mostre que o consumo de tempo da função *intercala* não é proporcional ao número de comparações entre elementos do vetor.

9. A seguinte variante da função *intercala* faz a intercalação *in loco*, ou seja, sem usar vetor auxiliar. (Ela insere cada elemento de $v[q..r-1]$ em $v[p..q-1]$ como o [algoritmo Insertionsort](#).) Critique a função.

```
while (q < r) {
    int x = v[q], int i;
    for (i = q-1; i >= p && v[i] > x; --i)
        v[i+1] = v[i];
    v[i+1] = x;
    q++; }
```

10. A seguinte solução do problema da intercalação está correta? Quais os invariantes do `while`? (Observe que a função faz a intercalação *in loco*, ou seja, sem usar vetor auxiliar.) Qual o consumo de tempo?

```
int i, k, x;
i = p;
while (i < q && q < r) {
    if (v[i] >= v[q]) {
        x = v[q];
        for (k = q - 1; k >= i; --k)
            v[k+1] = v[k];
        v[i] = x;
        ++q; }
    ++i; }
```

11. DESAFIO: INTERCALAÇÃO IN LOCO. Invente um função de intercalação tão eficiente quanto *intercala* que resolva o problema *in loco*, ou seja, sem usar um vetor auxiliar.

12. Um algoritmo de intercalação é [estável](#) se não altera a posição relativa de elementos iguais. A função *intercala* discutida acima é estável? E se a comparação " $v[i] <= v[j]$ " for trocada por " $v[i] < v[j]$ "?

13. INTERCALAÇÃO DE LISTAS ENCADEADAS. Digamos (para efeito deste exercício) que uma LEC é uma [lista encadeada](#) (sem cabeça) que contém uma sequência crescente de números inteiros. Escreva uma função que intercale duas LECs, produzindo assim uma terceira. Sua função não deve alocar novas células na memória, mas reaproveitar as células das duas listas dadas.

14. UNIÃO DE LISTAS ENCADEADAS. Digamos que uma LEC é uma [lista encadeada](#) (sem cabeça) que contém uma sequência *estritamente* crescente de números inteiros. (Portanto, uma LEC representa um *conjunto* de números.) Escreva uma função que faça a *união* de duas LECs. A lista resultante deve ser uma LEC e deve ser construída com as células das duas listas dadas.

Intercalação com sentinelas

[Sedgewick](#) escreve o algoritmo de intercalação de uma maneira mais interessante. Primeiro, copia o vetor $v[p..q-1]$ para o espaço de trabalho $w[0..q-p-1]$; depois, copia $v[q..r-1]$ para o espaço $w[q-p..r-p-1]$ *em ordem inversa*. Com isso, a metade esquerda de w serve de [sentinela](#) para a metade direita, e vice-versa, durante o processo de intercalação. Assim, não há necessidade de verificar, a cada iteração, as condições de fronteira $i < q-p$ e $j \geq q-p$.

```
// Esta função recebe vetores crescentes
// v[p..q-1] e v[q..r-1] e rearranja
// v[p..r-1] em ordem crescente.

static void
```

```

s_intercala (int p, int q, int r, int v[])
{
    int i, j, *w;
    w = malloc ((r-p) * sizeof (int));

    for (i = p; i < q; ++i) w[i-p] = v[i];
    for (j = q; j < r; ++j) w[r-p+q-j-1] = v[j];
    i = 0; j = r-p-1;
    for (int k = p; k < r; ++k)
        if (w[i] <= w[j]) v[k] = w[i++];
        else v[k] = w[j--];
    free (w);
}

```

Tal como a versão anterior, esta consome tempo proporcional ao tamanho do vetor $v[p..r-1]$.

Exercícios 2

1. Discuta o seguinte código alternativo de `s_intercala`:

```

for (i = 0, k = p; k < q; ++i, ++k)
    w[i] = v[k];
for (j = r-p-1, k = q; k < r; --j, ++k)
    w[j] = v[k];
i = 0; j = r-p-1;
for (k = p; k < r; ++k)
    if (w[i] <= w[j]) v[k] = w[i++];
    else v[k] = w[j--];

```

2. [Sedgewick 8.6] Mostre que a função `s_intercala` discutida acima não é [estável](#). Que modificações é preciso introduzir para que ela se torne estável?

Algoritmo Mergesort

O algoritmo Mergesort usa a estratégia da [divisão-e-conquista](#) para ordenar o vetor dado. A fase da divisão é simples: basta quebrar o vetor em dois. A fase da conquista foi implementada acima pela função `intercala`.

A função [recursiva](#) abaixo rearranja o vetor $v[p..r-1]$ em ordem crescente. A base da recursão é formada pelas instâncias em que $p \geq r-1$; nessas instâncias, o vetor tem no máximo 1 elemento e portanto não é preciso fazer nada para deixar o vetor fique em ordem crescente.

```

// A função mergesort rearranja o vetor
// v[p..r-1] em ordem crescente.

```

```

void
mergesort (int p, int r, int v[])
{
    if (p < r-1) {
        int q = (p + r)/2;
        mergesort (p, q, v);
        mergesort (q, r, v);
        intercala (p, q, r, v);
    }
}

```

(Notas: 1. Você pode trocar `intercala` por `s_intercala` na linha 5 pois essas duas funções são equivalentes. 2. O resultado da divisão por 2 na expressão $(p+r)/2$ é automaticamente [truncado](#). Por exemplo, $(3+6)/2$ vale 4.)

Se $p < r-1$, a [instância](#) $v[p..r-1]$ do problema é reduzida ao par de instâncias $v[p..q-1]$ e $v[q..r-1]$. Essas duas instâncias são estritamente menores que a instância original, uma vez que $q < r$ e $q > p$ (confira!) no fim da linha 2. Assim, por hipótese de indução, o vetor $v[p..q-1]$ estará em ordem crescente no fim da linha 3 e o vetor $v[q..r-1]$ estará em ordem crescente no fim da linha 4. Portanto, no fim da linha 5, o vetor $v[p..r-1]$ estará em ordem crescente, graças à [operação de intercalação](#).

0	1	2	3	4	5	6	7	8	9	10
111	999	222	999	333	888	444	777	555	666	555
111	999	222	999	333	888	444	777	555	666	555
111	999	222	999	333	888	444	777	555	666	555
⋮										
111	999	222	333	999	444	777	888	555	555	666
111	222	333	999	999	444	555	555	666	777	888
111	222	333	444	555	555	666	777	888	999	999

Para rearranjar em ordem crescente um vetor $v[0..n-1]$, como quer a [formulação original do problema](#), basta executar mergesort (θ , n , v).

Exercícios 3

1. Mostre que $p < q < r$ no fim da linha 2 de mergesort.
2. Que acontece se trocarmos " $(p+r)/2$ " por " $(p+r-1)/2$ " no código da função mergesort? Que acontece se trocarmos " $(p+r)/2$ " por " $(p+r+1)/2$ "?
3. Submeta um vetor indexado por 1..4 à função mergesort. Teremos a seguinte sequência de invocações da função (observe a indentação):

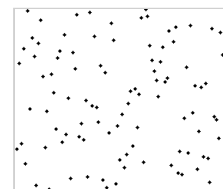
```
mergesort (1,5,v)
    mergesort (1,3,v)
        mergesort (1,2,v)
        mergesort (2,3,v)
    mergesort (3,5,v)
        mergesort (3,4,v)
        mergesort (4,5,v)
```

Repita o exercício com um vetor indexado por 1..5.

4. PEGADINHA. Quais são os invariantes da função mergesort?
5. TESTE DE CORREÇÃO. Escreva um programa para testar, experimentalmente, a correção de sua implementação do algoritmo Mergesort. (Veja o [exercício análogo para o Insertionsort](#).)
6. A ordenação produzida pela função mergesort é [estável](#)?
7. OVERFLOW. Se o tamanho do vetor estiver próximo de `INT_MAX`, a execução da função mergesort pode descarrilar na linha 2 em virtude de um [overflow](#) aritmético. Como evitar isso?

Animações do Mergesort

A animação à direita (copiada da [Wikipedia](#)) mostra a ordenação de um vetor $v[0..99]$ que contém uma permutação aleatória de $0..99$. (Veja uma [versão mais lenta](#) da animação.) Cada elemento $v[i]$ é representado pelo ponto de coordenadas $(i, v[i])$.



Há muitas outras animações do Mergesort na rede WWW. Veja algumas:

- [Comparison Sorting Algorithms](#), de David Galles (University of San Francisco).
- [Sorting Algorithms Animations](#).
- animação [15 sorting algorithms in 6 minutes](#), de Timo Bingmann, no YouTube.
- [Merge-sort com dança folclórica da Transilvânia](#), Universidade Sapiientia (Romênia).

Exercícios 4

1. A seguinte função promete rearranjar $v[p..r-1]$ em ordem crescente. A função está correta?

```
void mergesort1 (int p, int r, int v[]) {
    if (p < r-1) {
        int q = (p + r) / 2;
        mergesort1 (p, q, v);
        mergesort1 (q, r, v);
        intercala (p, q+1, r, v); } }
```

2. A seguinte função promete rearranjar $v[p..r-1]$ em ordem crescente. A função está correta?

```
void mergesort2 (int p, int r, int v[]) {
    if (p < r) {
        int q = (p + r) / 2;
        mergesort2 (p, q, v);
        mergesort2 (q, r, v);
        intercala (p, q, r, v); } }
```

3. Esta função está correta? Ela promete rearranjar $v[p..r-1]$ em ordem crescente.

```
void mergesort3 (int p, int r, int v[]) {
    if (p < r-1) {
        int q = (p + r - 1) / 2;
        mergesort3 (p, q, v);
        mergesort3 (q, r, v);
        intercala (p, q, r, v); } }
```

4. Esta função rearranja $v[p..r-1]$ em ordem crescente? E se trocarmos “ $(p+r)/2$ ” por “ $(p+r+1)/2$ ”?

```
void mergesort4 (int p, int r, int v[]) {
    if (p < r-1) {
        int q = (p + r) / 2;
        mergesort4 (p, q-1, v);
        mergesort4 (q-1, r, v);
        intercala (p, q-1, r, v); } }
```

5. Esta função rearranja $v[p..r-1]$ em ordem crescente?

```
void mergesort5 (int p, int r, int v[]) {
    if (p < r-1) {
        q = r - 2;
        mergesort5 (p, q, v);
        if (v[r-2] > v[r-1]) {
            int t = v[r-2];
            v[r-2] = v[r-1];
            v[r-1] = t; }
        intercala (p, q, r, v); } }
```

6. Esta função rearranja $v[p..r-1]$ em ordem crescente?

```
void mergesort6 (int p, int r, int v[]) {
    if (p < r-1) {
        q = r - 1;
        mergesort6 (p, q, v);
        intercala (p, q, r, v); } }
```

7. Suponha que sua biblioteca tem uma função `mrg(p, q, r, v)` que recebe um vetor v tal que $v[p..q]$ e $v[q+1..r-1]$ são crescentes e rearranja o vetor de modo que $v[p..r-1]$ fique crescente. Use `mrg` para implementar o algoritmo Mergesort.
8. Suponha que sua biblioteca tem uma função `interc(v, p, q, r)` que recebe um vetor v tal que $v[p..q-1]$ e $v[q..r-1]$ estão em ordem crescente e rearranja o vetor de modo que $v[p..r-1]$ fique em ordem crescente. (Qual o menor valor de q que `interc` deve aceitar? Qual o maior valor?) Use `interc` para escrever uma função `mrgsrt(v, p, r)` que rearranje um vetor $v[p..r]$ em ordem crescente.

Desempenho do algoritmo Mergesort

Aplique a função `mergesort` a um vetor $v[0..n-1]$. O tamanho do vetor é reduzido à metade a cada passo da recursão. Na primeira “rodada”, a instância original do problema é reduzida a duas menores: $v[0..n/2-1]$ e $v[n/2..n-1]$. Na segunda “rodada”, temos quatro instâncias:

$v[0..n/4-1]$, $v[n/4..n/2-1]$, $v[n/2..3n/4-1]$ e $v[3n/4..n-1]$.

E assim por diante, até que, na última “rodada”, cada instância tem no máximo 1 elemento. O número total de “rodadas” é aproximadamente $\log n$ (portanto também aproximadamente $\lg(n)$).

Em cada “rodada”, a função `intercala` executa $2n$ [movimentações](#) de elementos do vetor $v[0..n-1]$. Assim, o número total de movimentações para ordenar $v[0..n-1]$ é aproximadamente

$$2n \log n .$$

É fácil constatar que o consumo de *tempo* da função `mergesort` é proporcional ao número total de movimentações, e portanto proporcional a

$$n \log n .$$

Diz-se que o algoritmo é [linearítmico](#). O número $n \log n$ cresce muito mais devagar que n^2 e apenas um pouco mais rapidamente que n . Assim, se um vetor de tamanho N exige T unidades de tempo, um vetor de tamanho $2N$ exigirá menos que $2.2 T$ unidades de tempo, desde que N seja maior que 2^{10} . Da mesma forma, um vetor de tamanho $4N$ exigirá menos que $4.4 T$ unidades de tempo, desde que N seja maior que 2^{20} . (Confira as contas!)

O consumo de tempo do Mergesort é proporcional a $n \log n$ enquanto o dos [algoritmos elementares](#) é proporcional a n^2 . Mas o fator de proporcionalidade é maior para o Mergesort, pois o código é mais complexo. Assim, o Mergesort só se torna realmente mais rápido que os algoritmos elementares quando n é suficientemente grande.

Exercícios 5

1. Como o consumo de tempo do seguinte fragmento de código varia com n ?

```
int c = 1;
for (int i = 0; i < n; i *= 2)
    for (int j = 1; j < n; ++j)
        c += 1;
```

2. VERSÃO “TRUNCADA”. Escreva uma versão do algoritmo Mergesort com *cutoff* para vetores pequenos: quando o vetor a ser ordenado tiver menos que M elementos, a ordenação passa a ser feita pelo [algoritmo Insertionsort](#). O valor de M pode ficar entre 10 e 20. (Esse truque é usado na prática porque o algoritmo Insertionsort é mais rápido que o

Mergesort puro quando o vetor é pequeno. O fenômeno é muito comum: algoritmos sofisticados são tipicamente mais lentos que algoritmos simplórios quando o volume de dados é pequeno.)

3. ALOCAÇÃO/DESALOCAÇÃO EXCESSIVOS. A função `mergesort` acima chama as funções `malloc` e `free` muitas vezes (as chamadas acontecem dentro de `intercala`). Escreva uma versão da função que contenha o código da função de intercalação e chame `malloc` uma só vez.
4. DESAFIO: MERGESORT IN LOCO. Invente uma implementação de Mergesort que façam o serviço *in loco*, ou seja, dispensem o uso de um vetor auxiliar.
5. ORDEM DECRESCENTE. Escreva uma versão recursiva do algoritmo Mergesort que reorganize um vetor `v[p..r-1]` em ordem decrescente. Sua função deve conter o código da intercalação (que deve começar com `v[p..q-1]` e `v[q..r-1]` decrescentes e terminar com `v[p..r-1]` decrescente).
6. A seguinte função recursiva pretende encontrar o valor de um elemento máximo do vetor `v[p..r]`, não necessariamente ordenado. É claro que o problema só faz sentido se $p \leq r$.

```
int max (int p, int r, int v[]) {
    if (p == r) return v[r];
    else {
        int q = (p + r)/2;
        int x = max (p, q, v);
        int y = max (q+1, r, v);
        if (x >= y) return x;
        else return y; } }
```

A função está correta? Ela é mais rápida que a função iterativa óbvia? Quantas vezes a função chama a si mesma? Suponha que `p` e `r` valem 0 e 6 respectivamente e mostre a sequência, devidamente indentada, das chamadas de `max`.

7. TESTE DE DESEMPENHO. Escreva um programa para cronometrar sua implementação do Mergesort. (Veja o [exercício análogo para o Insertionsort](#). Para o Mergesort, você pode fazer testes para uma sequência maior de valores de n , talvez $2^8, 2^9, \dots, 2^{29}, 2^{30}$.)

Versão iterativa do Mergesort

A versão iterativa do algoritmo Mergesort reorganiza um vetor `v[0..n-1]` em ordem crescente. A cada iteração, o algoritmo intercala dois blocos consecutivos de `b` elementos cada: o primeiro bloco com o segundo, o terceiro com o quarto, etc. A variável `b` assume os valores 1, 2, 4, 8, ...

```
// Esta função reorganiza o vetor
// v[0..n-1] em ordem crescente.

void
mergesort_i (int n, int v[])
{
    int b = 1;
    while (b < n) {
        int p = 0;
        while (p + b < n) {
            int r = p + 2*b;
            if (r > n) r = n;
            intercala (p, p+b, r, v);
            p = p + 2*b;
        }
        b = 2*b;
    }
}
```

A figura ilustra a iteração em que `b` vale 2:

0				p		p+b		p+2b		n-1
111	999	222	999	333	888	444	777	555	666	555

Há muitas animações e visualizações interessantes da versão iterativa do Mergesort:

- [Animação do Mergesort](#) produzida por Mike Bostock. [Dica de Yoshiharu Kohayakawa.] O vetor é representado por uma “vassoura”. Cada cerda da “vassoura” é um elemento do vetor e a inclinação da cerda é o valor do elemento. Seguem algumas apresentações autônomas da animação:
 - [Mergesort I](#). (Veja [o código](#).)
 - [Mergesort II](#). Os segmentos que estão sendo ordenados ficam em destaque. (Veja [o código](#).)
 - [Mergesort III](#). Uma visualização estática. (Veja [o código](#).)
- [Mergesort IV](#), por Mike Bostock. Uma animação/visualização com fitas coloridas que se entrelaçam. Recarregue para variar os dados. (Veja [o código](#).)

Exercícios 6

1. INVARIANTES. Quais são os invariantes do `while` externo na função `mergesort_i`? E os invariantes do `while` interno?
2. SEGMENTOS CRESCENTES. A função `mergesort_i` começa por quebrar o vetor original em segmentos de comprimento 1. Por que não começar com segmentos crescentes maximais? Exemplo: os segmentos crescentes maximais do vetor 1 2 3 0 2 4 6 4 5 6 7 8 9 são 1 2 3, 0 2 4 6 e 4 5 6 7 8 9. Explore esta ideia.

Exercícios 7

1. LISTAS ENCADEADAS. Escreva uma versão do algoritmo Mergesort que rearranje uma lista encadeada em ordem crescente. Sua função não deve alocar novas células na memória. Faça duas versões: uma recursiva e uma iterativa.
2. NÚMERO DE INVERSÕES. O *número de inversões* de um vetor $v[0..n-1]$ é o número de pares ordenados (i, j) tais que $0 \leq i < j < n$ e $v[i] > v[j]$. Escreva uma função que calcule o número de inversões de um vetor dado. O consumo de tempo de sua função deve ser proporcional a $n \log n$ no pior caso.
3. DISTÂNCIA TAU DE KENDALL. Suponha dadas duas [permutações](#), digamos $x[0..n-1]$ e $y[0..n-1]$, de um mesmo conjunto de números. A *distância tau* entre x e y é o número de pares de elementos do conjunto que estão em ordem diferente em x e y , ou seja, a cardinalidade do conjunto $X - Y$ onde X é o conjunto de todos os pares $(x[i], x[j])$ tais que $i < j$ e Y é o conjunto de todos os pares $(y[i], y[j])$ tais que $i < j$. (A definição não é assimétrica como parece pois os conjuntos $X - Y$ e $Y - X$ têm a mesma cardinalidade.) Escreva uma função eficiente que calcule a distância tau entre x e y .

Veja o verbete [Merge sort](#) na Wikipedia.

Veja [Mergesort no Cprogramming.com](#).

Veja o algoritmo [Timsort](#), baseado em ideias semelhantes às do Mergesort.

Atualizado em 2019-03-01

<https://www.ime.usp.br/~pf/algoritmos/>

© Paulo Feofiloff

[DCC-IME-USP](#)